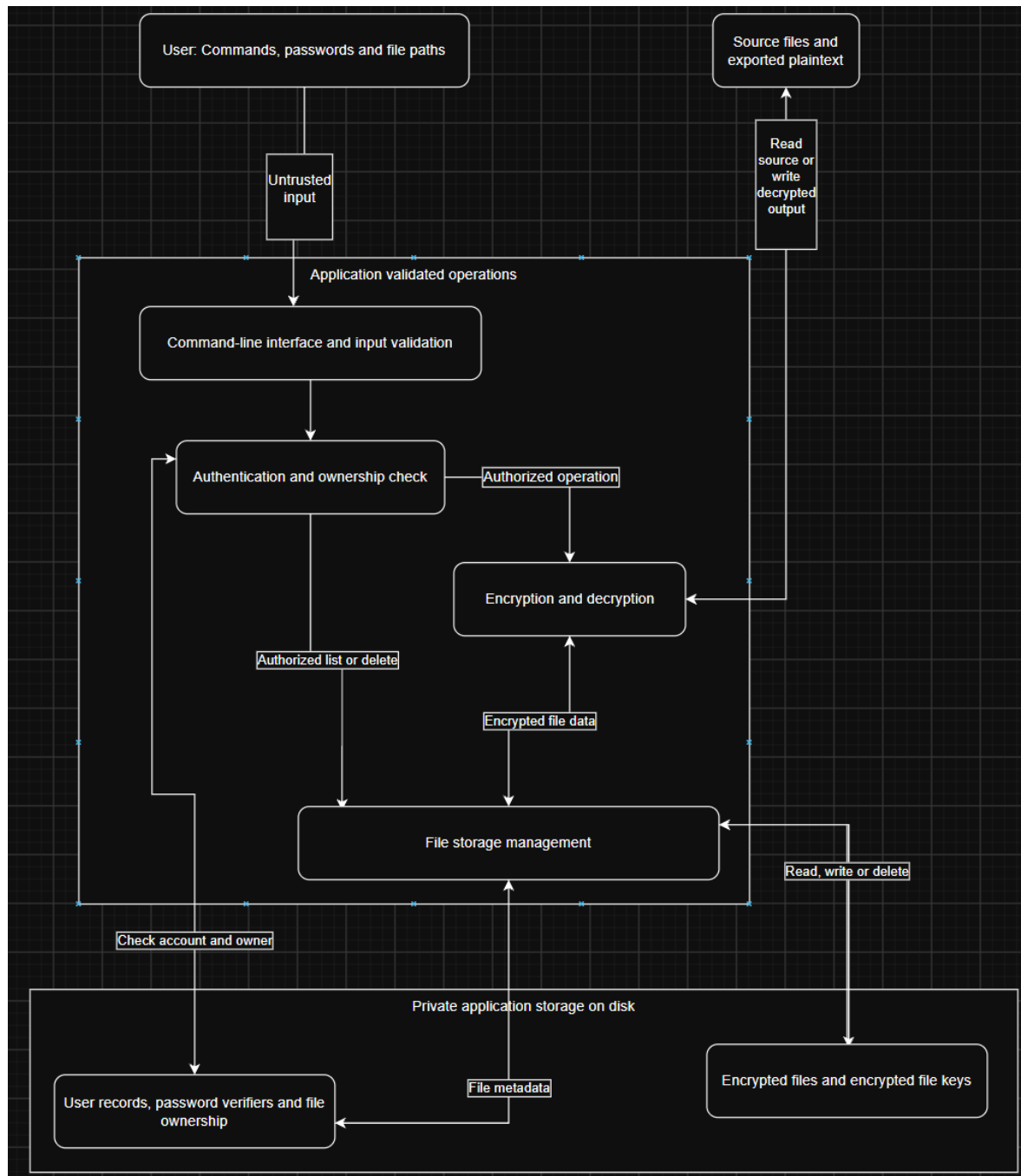


Secure File Encryption and Management System

1. System architecture

The project will be a Java command-line application. Users will enter a password to access the tool and will be able to encrypt, list, decrypt and delete their own stored files.



Components and data flow: The command-line interface receives commands and validates input. Authentication checks the password, and ownership checks ensure that users access only their own stored files. The encryption component encrypts source files or decrypts stored files. Storage management handles the encrypted files and ownership records.

File and key storage: Encrypted files, protected keys and user records will be stored in a private application directory with restricted permissions. User records will contain salted password hashes, not plaintext passwords. The proposed key design is to store file keys encrypted under a key derived from the user's password. Unencrypted keys will be held in memory only while needed.

Trust boundaries: User input and source files are untrusted. The application must validate input and check authentication and ownership before granting access. The private storage boundary is protected by operating-system permissions. These checks assume a trusted operating system.

2. Threat model

Threat	Planned mitigation
A malicious filename accesses files outside the managed directory.	Validate file paths and ensure managed-file operations stay inside the intended directory.
A symbolic link redirects an operation to another file.	Reject symbolic links in managed-file operations and use file-opening methods that avoid a gap between checking and using a file.
A user accesses or deletes another user's stored files.	Authenticate users and check ownership on every file operation. Deny access if a check fails.
Someone obtains stored passwords.	Store salted password hashes instead of plaintext passwords, as required by the project.
Someone reads or changes stored file contents without permission.	Encrypt stored files, check their integrity and restrict access using file permissions.
An encryption key is exposed.	Store file keys in encrypted form and never write unprotected keys or passwords to logs.
A file operation overwrites an existing file.	Create output files in a way that fails if the destination already exists.
Invalid or excessive input causes errors or memory problems.	Check input type, format and length, and use bounded buffers when processing files.

An operation fails and leaves resources open or incomplete output.	Handle errors, close files reliably and remove incomplete output where safe. Report a generic error without exposing secrets.
--	---

3. Language and proposed libraries

Choice	Reason
Java	I have previous experience with Java. Array bounds checks and automatic memory management help reduce common memory errors.
Java cryptography APIs (JCA/JCE)	Use existing library implementations for file encryption and salted password hashing instead of implementing cryptographic algorithms myself.
Java file APIs (java.nio.file)	Read and write files, validate paths and manage permissions.
Java Console and try-with-resources	Read passwords without displaying them and close files even when errors occur. Use mutable arrays for secrets and clear them after use.

No database library is selected at this stage. User and ownership records will be stored locally in the private application directory, the exact format remains to be decided.